

Docker and PHP

Run a PHP file, then a small API.

Learning Goals

- Run a PHP file and a small API in a container, using a mounted folder.
- Read a port mapping such as `-p 8000:8000` (host : container).
- Build an image from a Dockerfile and run a container from it.

Open the Example Folder

Examples are under `module3/code/1_Docker/2. Docker and PHP/` :

- `1_test/` : run `test.php` once.
- `2_api/` : run `api.php` with PHP's development server.
- `3_dockerfile/` : build an image containing the API.

Copy the example you want to a practice directory.

Run Your PHP File (1_test)

From the folder containing `test.php` :

```
docker run --rm -v "$PWD:/app" -w /app \
  php:8.2-cli php test.php
```

- `-v "$PWD:/app"` mounts (not copies) your current host folder onto `/app` inside the container, so `test.php` is visible there without being duplicated.
- `-w /app` sets `/app` as the container's working directory.
- `php test.php` then runs the mounted file directly inside the container.

Start the API (2_api)

From the folder containing `api.php` :

```
docker run --rm -p 8000:8000 -v "$PWD:/app" -w /app \
  php:8.2-cli php -S 0.0.0.0:8000
```

- The php server (`php -S 0.0.0.0:8000`) is run on the container.
- The PHP is run on an isolated Docker container, but we can access the PHP server without any problems.
- Stop the server with **Ctrl+C**.

In another terminal, run `curl -i`
`http://localhost:8000/api.php` .

```
HTTP/1.1 200 OK
Host: localhost:8000
Date: Thu, 01 Oct 2026 17:00:31 GMT
Connection: close
X-Powered-By: PHP/8.4.26
Content-Type: application/json

{
    "message": "Hello from Docker PHP!",
    "php_version": "8.4.26",
    "timestamp": 1790874031,
    "server": "Docker Container"
}%
```

Read the Port Mapping

```
-p 8000:8000  
host : container
```

Your browser uses `localhost:8000` on the host.

The server listens on port 8000 inside the container.

If the host port is busy, use `-p 8001:8000` and browse port 8001.

Package the API in an Image (3_dockerfile)

The `3_dockerfile/` example uses the Dockerfile to specify the options.

```
FROM php:8.4-cli
WORKDIR /app
COPY . .
EXPOSE 8000
CMD ["php", "-S", "0.0.0.0:8000"]
```

- `COPY` includes files at build time;
- `EXPOSE` documents the port.

Build

From the folder containing the Dockerfile:

```
docker build -t my-php-app .
```

- `-t` (`--tag`) gives the built image the name `my-php-app` ; without an explicit tag, Docker uses `my-php-app:latest` .
- `.` sets the build context to the current directory, where Docker looks for the `Dockerfile` and files used by `COPY` .

Run

```
docker run --rm -p 8000:8000 my-php-app
```

- `my-php-app` is the image name assigned by `docker build -t my-php-app .`
- It is used by `docker run`; it is not the container name.

Test

Use curl or open a web browser:

```
curl http://localhost:8000/
```

```
{  
  "message": "Hello from Docker PHP!",  
  "php_version": "8.4.26",  
  "timestamp": 1790883871,  
  "server": "Docker Container"  
}
```